Herzlich willkommen!

Parameter	Kursinformationen
Veranstaltung:	`Erhebung, Analyse und Visualisierung digitaler Daten`
Semester	`Wintersemester 2026/27`
Hochschule:	`Technische Universität Freiberg`
Inhalte:	`Motivation, Lernziele, Arbeitsweise, Semesterplan, Prüfung, Organisation`
Link auf Repository:	https://github.com/TUBAF-IfI-LiaScript/VL_EAVD/blob/master/00_Motivation.md
Autoren	Sebastian Zug & Bernhard Jung

Zum Nacharbeiten: die Ansicht „Lehrbuch“. Im Hörsaal zeigen die Folien nur die Kernbotschaften. Die Erklärungen dazu stehen im Lehrbuch: oben rechts auf das Symbol mit den Schiebereglern klicken und „Lehrbuch“ wählen.

Prof. Dr. Sebastian Zug

Professur für Softwareentwicklung und Robotik

sebastian.zug@informatik.tu-freiberg.de

Prof. Dr. Bernhard Jung

Professur für Virtuelle Realität und Multimedia

bernhard.jung@informatik.tu-freiberg.de

Warum sind Sie hier?

Was motiviert Sie, an dieser Veranstaltung teilzunehmen?

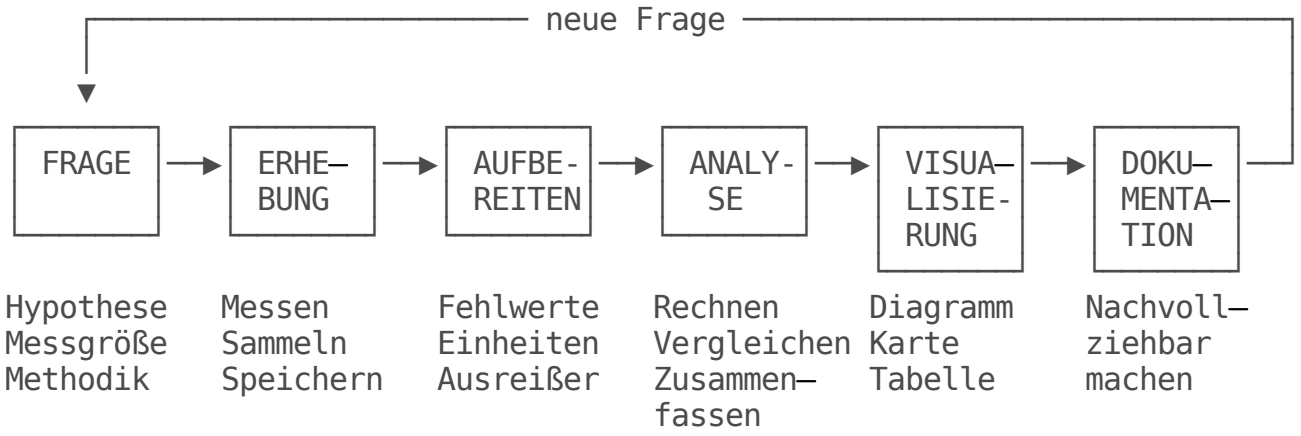
Texteingabe ...

Was verbinden Sie mit dem Begriff „Programmieren“?

Texteingabe …

Warum ist die Veranstaltung zentral?

In Ihrem Fach werden Sie Messreihen auswerten, Analysendaten sortieren und Ergebnisse darstellen. Die Daten durchlaufen dabei immer wieder dasselbe Muster:



Der Pfeil zurück ist der wichtigste. Fast nie führt der erste Durchlauf zum Ziel — auswerten heißt, diese Schleife mehrfach zu drehen.

Von Hand oder mit einem Programm? Diese Frage stellt sich bei jedem Durchlauf.

Die Daten reichen nicht, die Frage war zu unscharf, oder das Ergebnis wirft eine neue Frage auf. Der Name der Veranstaltung nennt drei der Schritte — Erhebung, Analyse und Visualisierung. Ein Programm erledigt sie jedes Mal wieder, nachvollziehbar und ohne Tippfehler; die Tabellenkalkulation verlangt jedes Mal neue Handarbeit.

Lernziele

Am Ende des Semesters können Sie eine wissenschaftliche Fragestellung auf eine Datenverarbeitung abbilden — von der Rohdatei bis zur belegten Aussage. Dafür brauchen Sie beides: das Handwerkszeug und seine Anwendung.

Sie können …	Handwerkszeug	Anwendung auf Daten
entwickeln	ein eigenes Programm aus Schleifen, Bedingungen und Funktionen aufbauen	eine vollständige Auswertung schreiben — einlesen, bereinigen, auswerten, darstellen
beurteilen	einschätzen, ob ein Programm verständlich und wartbar geschrieben ist	einschätzen, ob ein Ergebnis belastbar ist und wann eine Darstellung in die Irre führt
analysieren	fremden Python-Code lesen und nachvollziehen, was er tut	einen unbekannten Datensatz erschließen: Was steht drin, was fehlt darin?
anwenden	Variablen, Datentypen, Schleifen und Funktionen zur Lösung einer Aufgabe nutzen	die Werkzeuge der Datenanalyse auf eigene Messreihen und Analysendaten übertragen
verstehen	erklären, was Variable, Datentyp, Schleife, Funktion und Modul bedeuten	erklären, warum Daten aufbereitet werden müssen, bevor man mit ihnen rechnet
erinnern	die Syntax der Grundbausteine benennen	die Grundbegriffe benennen: Algorithmus, Fehlwert, Bibliothek, DataFrame

Drei Fähigkeiten, die über den Kurs hinaus tragen

1. **Sie erkennen, wann sich Programmieren lohnt** — und wann eine Tabellenkalkulation genügt. Beides zu wissen ist mehr wert, als eine Sprache zu kennen.
2. **Sie können Ergebnisse hinterfragen** — die eigenen und die anderer. Woher kommen die Daten, was fehlt darin, was trägt die Aussage wirklich?
3. **Sie können sich selbst weiterhelfen** — Fehlermeldungen lesen, Dokumentation nutzen, Beispiele übertragen. Kein Kurs deckt ab, was Sie später brauchen werden.

Was ausdrücklich nicht das Ziel ist: Sie werden keine Software-Entwickler. Kleine Vorhaben können Sie aber selbst umsetzen, größere zumindest einordnen.

Praktische Inhalte

Wann erscheinen Studierende in der Vorlesung? Gibt es Muster — der frühe Vogel, rechtzeitig aber knapp, in den ersten zehn Minuten passiert ohnehin nichts?

Ein kleiner Roboter an der Hörsaaltür zählt, wer hereinkommt und wer hinausgeht — auch heute, während Sie hereinkommen. Ein Auszug aus einem Probelauf:

zaehlungen_2026-10-15.csv

```
zeit;richtung
2026-10-15T16:42:31;raus
2026-10-15T16:42:32;unklar
2026-10-15T16:42:36;raus
2026-10-15T16:42:40;rein
2026-10-15T16:43:04;unklar
2026-10-15T16:43:08;rein
```

- Wie kommen die Daten zustande? Und zählt der Roboter richtig?
- Wie macht man aus diesen Textzeilen etwas, womit man rechnen kann?
- Wie beantwortet man die Frage — heute, und jede Woche aufs Neue?

Eine einfache Frage, echte Messdaten — und schon stecken wir mitten in den Themen dieses Semesters. Am Ende dieser Vorlesung kommen wir darauf zurück.

Ein Blick voraus: Körner im Dünnschliff

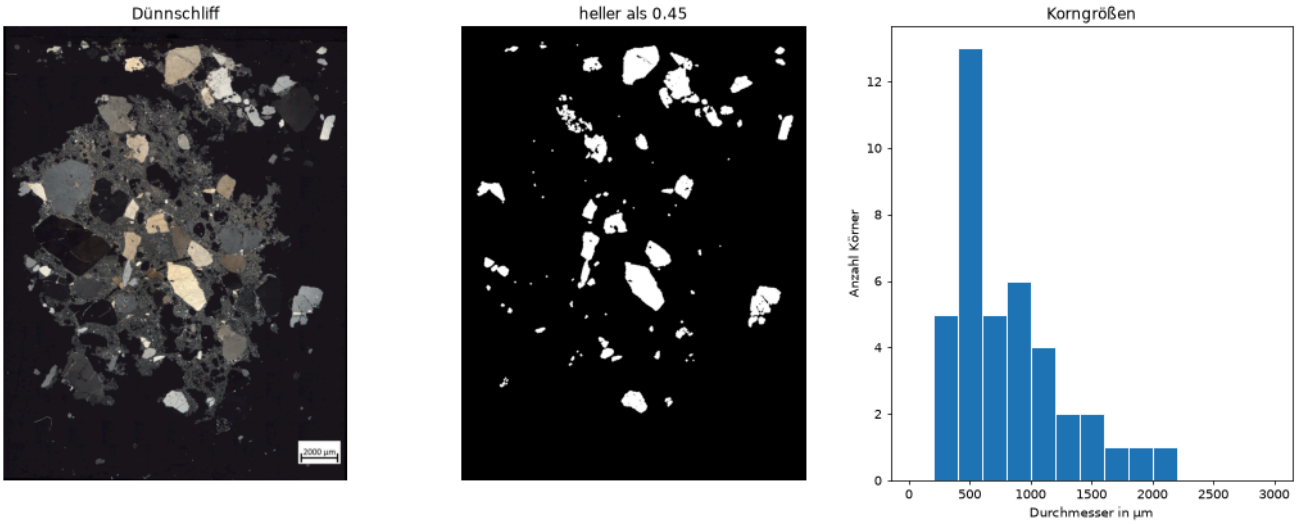
Wie groß sind die Körner in diesem Gestein — und wie viel Fläche nehmen sie ein?

Ein **Dünnschliff** ist eine Gesteinsscheibe, so dünn geschliffen, dass Licht hindurchscheint. Sie müssen heute keine Zeile des Programms verstehen — achten Sie nur auf die Kommentare rechts:

duennschliff.py

```
grau[900:, 640:] = 0            # Maßstab unten rechts ausblenden
UM_PRO_PIXEL = 2000 / 80        # Maßstab: 2000 µm sind 80 Pixel
SCHWELLE = 0.45     # Annahme: Ein Korn ist heller als die Schwelle
hell = ndimage.binary_opening(grau > SCHWELLE, iterations=2)  # Krümel weg
koerner, _ = ndimage.label(hell)              # zusammenhängende Flecken
flaeche = np.bincount(koerner.ravel())[1:]  # Pixel je Fleck
flaeche = flaeche[flaeche >= 150]             # sehr kleine Flecken weg
durchmesser = 2 * np.sqrt(flaeche / np.pi) * UM_PRO_PIXEL
```

Ergebnis: **40 Körner**, mittlerer Durchmesser (Median) **657 µm**, zusammen rund **13** % der Probenfläche. Bild: Fundus von Prof. Heide, TU Bergakademie Freiberg.



Der Schliff ist etwa 0,03 mm dick. Unter dem Mikroskop liegt er zwischen zwei gekreuzten Polarisationsfiltern; die Mineralkörner erscheinen dann je nach Mineral und Lage hell, farbig oder dunkel. Das Programm sucht die hellen Flecken, zählt sie und rechnet ihre Fläche über den Maßstab in Mikrometer um.

Jede Zeile eine Entscheidung

Das Programm läuft in wenigen Sekunden — aber jede Zeile enthält eine Entscheidung:

Im Code steht …	Die Annahme dahinter	Was schiefgeht
`SCHWELLE = 0.45`	Ab dieser Helligkeit beginnt ein Korn	Bei 0,35 sind es 48 Körner auf 21 % der Probe, bei 0,55 nur 32 auf 11 %
`grau > SCHWELLE`	Körner sind hell	Körner, die gerade dunkel erscheinen, fehlen ganz
`ndimage.label`	Körner berühren sich nicht	Zwei aneinanderliegende Körner zählen als eines
`grau[900:, 640:] = 0`	Der Maßstab ist kein Korn	Ohne diese Zeile wird der Maßstabsbalken als Korn mitgezählt
`UM_PRO_PIXEL = 2000 / 80`	80 Pixel entsprechen 2000 µm	Wer den Maßstab falsch abliest, verschiebt alle Durchmesser

Zum Ausprobieren: Das vollständige Programm liegt unter `examples/00_Einfuehrungsbeispiele/Duennschliff/` und läuft in Visual Studio Code.

Die Bausteine dafür lernen Sie im Semester: Bilder als Tabellen aus Zahlen und Masken in Vorlesung 10, Histogramme in Vorlesung 13.

Und was ist mit Ihnen

Aus welchem Studiengang kommen Sie?

Texteingabe …

Welche Daten würden Sie gern analysieren?

Texteingabe …

Welche Programmiererfahrungen haben Sie?

- ◯ Keine, ich habe noch nie programmiert.
- ◯ Ein bisschen, ich habe schon mal ein Programm geschrieben.
- ◯ Viel, ich habe schon mehrere Programme geschrieben.

Wie diese Vorlesung funktioniert

Die Frage kommt zuerst

Jede Vorlesung beginnt mit einer Frage an echte Daten. Die Sprachkonzepte kommen dann, wenn man sie zur Beantwortung braucht.

```
statt:    "Heute beschäftigen wir uns mit Schleifen."
sondern:  "Wir müssen 48.000 Zeilen durchgehen. Wie machen wir das?"
                                  ↓
                          Dafür gibt es Schleifen.
```

> Wer weiß, *wozu* ein Konzept dient, behält es.

Wer ein Konzept als Vokabel lernt, vergisst es nach der Klausur. Um einige Grundlagen kommen wir trotzdem nicht herum — aber immer mit Blick darauf, wofür wir sie brauchen.

Ein Datensatz durch das ganze Semester

Roter Faden ist die Klimamessreihe des Deutschen Wetterdienstes vom **Fichtelberg** (1213 m): Tageswerte seit 1890, fast 48.000 Zeilen, frei verfügbar — und mit allen Widrigkeiten echter Daten.

produkt_klima_tag_18900801_20251231_01358.txt

```
STATIONS_ID;MESS_DATUM;QN_3;  FX;  FM;QN_4; RSK;RSKF; SDK;SHK_TAG;  NM; VPM
  TMK; UPM; TXK; TNK; TGK;eor
      1358;19101210;-999;-999;-999;    1;   0.0;   0;-999;  90;   3.3;   7
        863.60;    3.8;   90.00;    5.4;    1.0;-999;eor
      1358;19151001;-999;-999;-999;    1;   2.0;   1;-999;   0;   8.0;   7
        873.70;    2.0;  100.00;    2.8;    1.0;-999;eor
```

Fehlwerte als `-999`, kryptische Spaltennamen, Datumsangaben ohne Trennzeichen — und zwischen diesen beiden Zeilen fehlen fast fünf Jahre.

Dass dieser Datensatz „schmutzig“ ist, ist kein Nachteil, sondern der Punkt. An sauberen Datensätzen lernt man nichts über Datenanalyse.

Dazu kommen Beispiele aus Ihren Fächern — Analysendaten, Bohrprofile, Pegelstände, Rohstoffpreise.

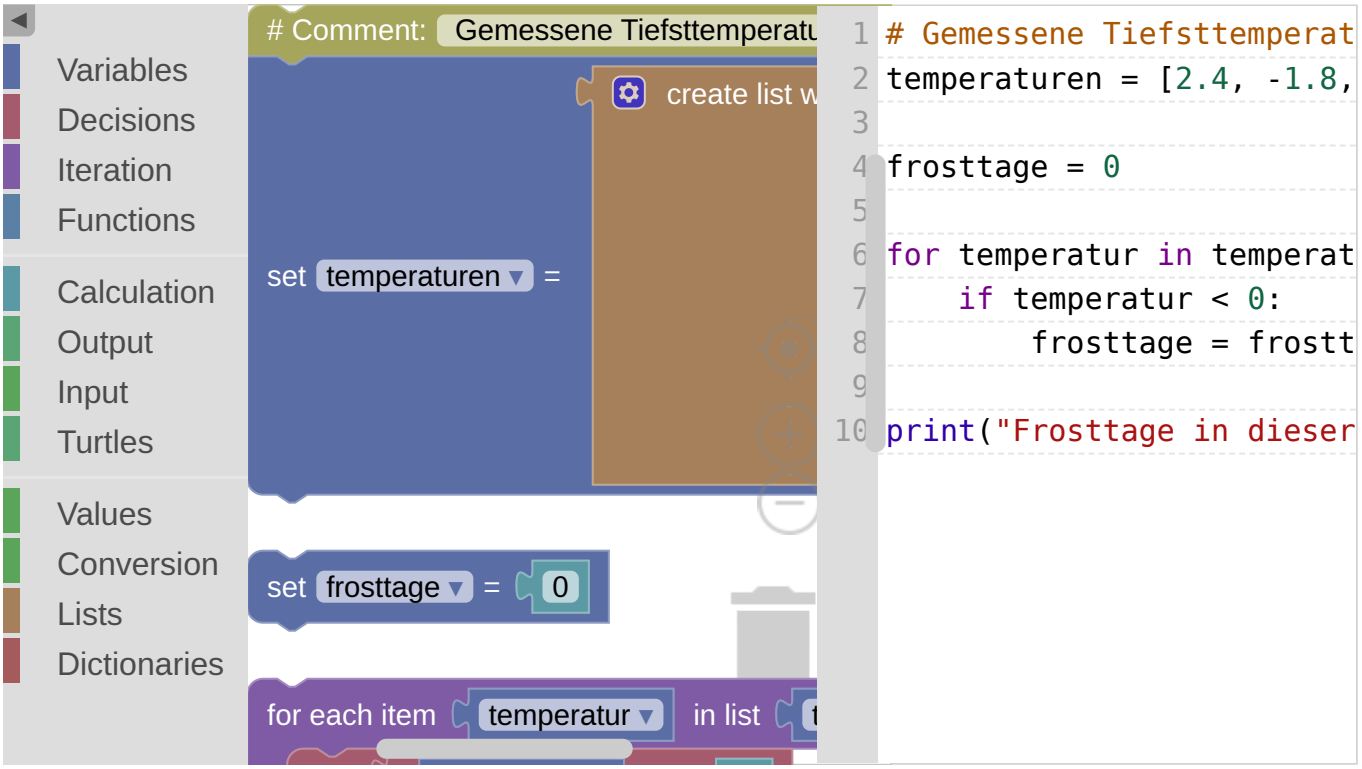
Ihre Antworten auf die Umfrage „Und was ist mit Ihnen“ helfen uns bei der Auswahl.

Blöcke oder Text

> **Caution**
>
> Das Laden der Beispiele dauert im Browser ein paar Sekunden. Bitte warten Sie, bis der Editor erscheint.

Die ersten Beispiele können Sie wahlweise als **Blöcke** zusammenschieben oder als **Text** tippen. Das sind nicht zwei Sprachen, sondern zwei Ansichten auf dasselbe Programm.

Eine Woche mit Tiefsttemperaturen — gezählt werden die **Frosttage**, an denen das Minimum unter 0 °C lag. Legen Sie sich fest, **bevor** Sie das Programm ausführen: Wie viele sind es?



Nutzen Sie die Schaltflächen über dem Editor, um zwischen Blöcken und Text zu wechseln. In Vorlesung 03 verstehen Sie jede dieser Zeilen.

Live Hacking und typische Fehlvorstellungen

- **Live Hacking** (ab Vorlesung 04): Ein Programm entsteht vor Ihren Augen — inklusive Irrwegen, Fehlermeldungen und Korrekturen.
- **Typische Fehlvorstellung**: Annahmen, die zu Programmen führen, die ohne Fehlermeldung falsch rechnen. Sie geben vorher eine Vorhersage ab — und merken, wenn Sie danebenliegen.

Programmieren ist ein Prozess, kein Ergebnis; fertiger Code auf einer Folie verschweigt genau den Teil, den Sie lernen müssen. Die Fehlvorstellungen hat fast jede und jeder am Anfang.

Werkzeuge

Wann	Werkzeug	Wofür
ab heute	Beispiele direkt im Browser	ausprobieren ohne Installation
ab Vorlesung 03	Python auf Ihrem Rechner, Visual Studio Code	eigene Programme, Übungsaufgaben
ab Vorlesung 10	Visual Studio Code mit Zellen (`# %%`)	schrittweise Datenanalyse mit Diagrammen

> Die Beispiele im Browser sind gewöhnliches Python. Sie lassen sich eins zu eins auf Ihren Rechner übertragen.

Erst mühsam, dann mit Werkzeug

> **Die Wetterdatei einlesen: in Vorlesung 06 rund 50 Zeilen, in Vorlesung 11 eine Anweisung.**

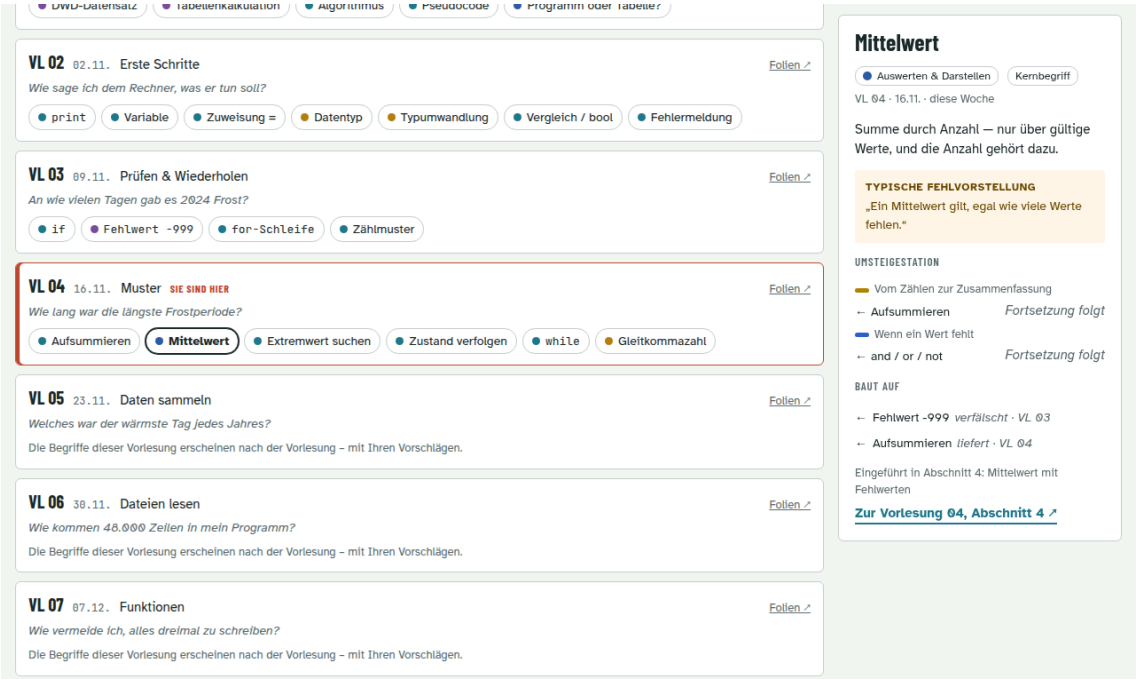
Wer den mühsamen Weg gegangen ist, versteht, *was* diese Anweisung abnimmt. Wer ihn übersprungen hat, steht hilflos da, sobald sie einmal nicht funktioniert.

Die Kurslandkarte

Wo stehen wir — und worauf baut auf, was heute dazukommt?

Die Kurslandkarte zeigt für jede Vorlesung die neuen Begriffe, und zu jedem Begriff

- eine Erklärung in einem Satz,
- die typische Fehlvorstellung, mit der viele am Anfang danebenliegen,
- worauf er aufbaut und wohin er führt,
- einen Link direkt zur passenden Folie.



Die Karte wächst mit dem Semester — nach jeder Vorlesung kommen ihre Begriffe dazu, auch Ihre Vorschläge.

Programmieren lernt man in Schichten: Jeder neue Begriff stützt sich auf frühere. Die Karte hält diesen Zusammenhang fest. Für kommende Vorlesungen stehen zunächst nur Termin und Leitfrage; wer neugierig ist, sieht mit „Ausblick: An" die vorbereiteten Begriffe blass. „Sie sind hier" richtet sich nach dem Datum. Neben der Liste gibt es eine Metrokarte, deren Linien rote Fäden durch das Semester sind — etwa „Wenn ein Wert fehlt" —, und eine Zeitleiste mit allen Begriffen und ihren Verbindungen.

Wann	Wofür
vor der Vorlesung	Was kommt heute dazu, und worauf baut es auf?
nach der Vorlesung	Kann ich jeden neuen Begriff in einem Satz erklären?
vor der Übung und der Klausur	Kenne ich die typische Fehlvorstellung — und warum sie falsch ist?

Offene Materialien

- **Im Browser, ohne Installation:** geschrieben mit LiaScript, Beispiele zum Ausführen und Verändern.
- **Präsentation im Hörsaal, Lehrbuch zum Nacharbeiten:** Umschalten über das Symbol mit den Schiebereglern oben rechts.
- **Frei verfügbar:** Lizenz CC BY 4.0, offen auf GitHub.
- **Lebendig:** Die Versionsnummer im Kopf jeder Vorlesung zeigt, wann sich etwas geändert hat.
- **Machen Sie mit:** Fehler melden Sie im OPAL-Forum oder per E-Mail. Wer zu einer Vorlesung beiträgt, wird dort als **Mitautorin oder Mitautor** genannt.

Die dritte Ansicht, „Folien“, zeigt die Erklärungen als Notiz unter jeder Folie. Die Materialien dürfen Sie nutzen, teilen und bearbeiten, solange Sie die Urheber nennen. Wer GitHub nutzt, kann Fehler auch als Issue melden oder Verbesserungen direkt als Änderung vorschlagen.

Semesterplan

Die Vorlesung findet montags von 16:15 bis 17:45 Uhr im Raum FOR-0270 statt. Vor Weihnachten legen wir die Grundlagen, danach geht es um Datenanalyse.

Termin	VL	Thema	Leitfrage
19.10.	00	Motivation, Organisation	Warum sollte ich programmieren lernen?
26.10.	01	Vom Problem zum Programm	Wo hört die Tabellenkalkulation auf?
02.11.	02	Erste Schritte in Python	Wie sage ich dem Rechner, was er tun soll?
09.11.	03	Prüfen und Wiederholen	An wie vielen Tagen gab es 2024 Frost?
16.11.	04	Muster in Messreihen	Wie lang war die längste Frostperiode?
23.11.	05	Daten sammeln	Welches war der wärmste Tag jedes Jahres?
30.11.	06	Dateien lesen	Wie kommen 48.000 Zeilen in mein Programm?
07.12.	07	Funktionen	Wie vermeide ich, alles dreimal zu schreiben?
14.12.	08	Demonstration: Datenerhebung mit MicroPython	Wo kommen die Daten eigentlich her?
04.01.	09	Programme strukturieren	Wie organisiere ich Code, der wächst?
11.01.	10	Bibliotheken: NumPy	Warum muss ich das Rad nicht neu erfinden?

18.01.	11	pandas I — Einlesen	48.000 Zeilen in einer Anweisung?
25.01.	12	pandas II — Aggregieren	Wie fasse ich Jahrzehnte zusammen?
01.02.	13	Visualisierung	Wie zeige ich, was ich gefunden habe?
08.02.	14	Datenqualität, Ausblick, Prüfungsvorbereitung	Kann ich meinem Ergebnis trauen?

Prüfung

Die Veranstaltung schließt mit einer **schriftlichen Prüfung** ab.

Eine Klausur prüft nicht, ob Sie eine Analyse *bauen* können, sondern ob Sie Code **lesen, verstehen, korrigieren und skizzieren** können. Darauf bereiten wir gezielt vor:

Kompetenz	**Wie sie geübt wird**
Code lesen und Ergebnis vorhersagen	Vorhersage-Fragen in jeder Vorlesung
Fehler finden	Live Hacking mit echten Irrwegen, typische Fehlvorstellungen
Code auf Papier schreiben	Übungsaufgaben ohne Rechner
Konzepte erklären	Skizzenaufgaben, Schwerpunkt Datenanalyse

Was in der Klausur vorkommt

- *Welchen Wert gibt das folgende Programm in Zeile x aus?*
- *Finden Sie alle syntaktischen und logischen Fehler im nachfolgenden Code.*
- *Schreiben Sie eine Funktion, die …*
- *Was macht* `groupby` *mit dieser Tabelle? Skizzieren Sie das Ergebnis.*
- *Warum ist diese Darstellung irreführend?*
- *Warum ist* `-999` *in diesem Datensatz ein Problem?*

Was nicht vorkommt: auswendig gelernte Bibliothekssyntax und die Inhalte der Mikrocontroller-Demonstration.

Vorlesung 14 enthält eine Prüfungsvorbereitung mit Musteraufgaben.

> Studierende der *Einführung in die Informatik* (7 LP) bearbeiten neben der Klausur eine praktische Programmieraufgabe. Thema und Umfang stimmen Sie mit den Übungsbetreuern ab.

Organisatorisches

Übungen. Die Übungen vertiefen jede Vorlesung an Programmieraufgaben zum Datensatz der Woche — ein Teil davon bewusst auf Papier, als Vorbereitung auf die Klausur. Sie beginnen in der zweiten Vorlesungswoche, also ab Montag, 26.10. Wählen Sie einen der vier Termine:

Tag	Uhrzeit	Raum
Montag	14:30–16:00	RAM-2119
Mittwoch	8:00–9:30	RAM-2119
Mittwoch	8:00–9:30	KKB-2097
Mittwoch	18:00–19:30	RAM-2119

In der ersten Übung richten wir gemeinsam Python und Visual Studio Code ein. Danach gehört jedes Übungsblatt zur Vorlesung der Woche: Sie versuchen sich bis zur Übung der Folgewoche daran, dort besprechen wir es.

Materialien. Alle Vorlesungen, Daten und Beispiele finden Sie unter github.com/TUBAF-IfI-LiaScript/VL_EAVD; wie Sie mitmachen können, steht im Abschnitt „Offene Materialien“.

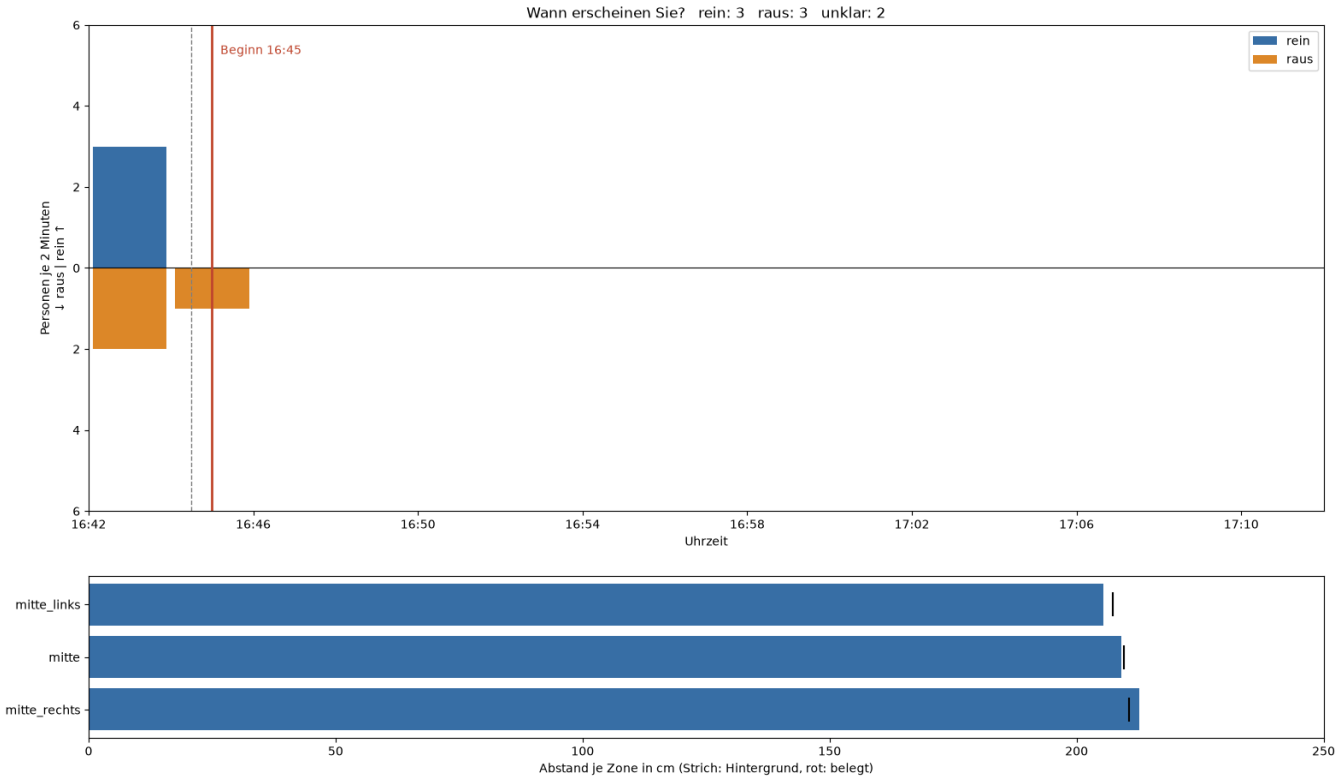
Zeitaufwand. 180 h: 60 h Präsenzzeit und 120 h Selbststudium für Vor- und Nachbereitung, Übungsaufgaben und Prüfungsvorbereitung. Ohne Programmiererfahrung sollten Sie eher mehr Zeit einplanen — in einer Arbeitsgruppe macht das deutlich mehr Spaß als allein.

Wie Sie zum Gelingen beitragen können

- Stellen Sie Fragen — in der Vorlesung, in der Übung, im OPAL-Forum.
- Bringen Sie Beispiele aus Ihrem Fach mit.
- Melden Sie Fehler und Verbesserungsvorschläge zu den Materialien — im OPAL-Forum, per E-Mail oder als Issue auf GitHub.

Zurück zum Personenzähler

Der Roboter, ein Arduino Alvik, misst mit Infrarotlicht die Entfernung in mehreren Zonen nebeneinander. Was deutlich näher ist als die Wand gegenüber, gilt als Person; wer zuerst links auftaucht, kommt herein. So sieht seine Live-Anzeige aus — hier aus einem Probelauf:



Welche Annahmen im Zählprogramm stecken

Die Einstellungen des Zählprogramms, in Python:

zaehler.py

```
# Auszug: die Einstellungen der Erkennung
ANTEIL = 0.8           # belegt, wenn näher als 80 % des Hintergrunds
NACHLAUF_MS = 400      # so lange müssen alle Zonen frei sein
MAX_DAUER_MS = 6000    # länger belegt: keine Person, neuer Hintergrund
REIN_VON = "links"     # wer zuerst links auftaucht, kommt herein
```

Sie müssen heute keine Zeile davon verstehen. Aber schon die Namen verraten, welche Annahmen darin stecken:

Im Code steht …	Die Annahme dahinter	Was schiefgeht
`ANTEIL = 0.8`	Eine Person ist deutlich näher als die Wand	Wer dicht an der gegenüberliegenden Seite vorbeigeht, wird übersehen
`NACHLAUF_MS = 400`	Zwischen zwei Personen ist die Tür kurz frei	Zwei dicht hintereinander zählen als eine
`MAX_DAUER_MS = 6000`	Niemand braucht länger als 6 s für die Tür	Wer in der Tür stehen bleibt, wird zur Wand — und nicht gezählt
`REIN_VON = "links"`	Wer hereinkommt, taucht zuerst links auf	Wer in der Tür umkehrt, zählt als „rein“ oder „unklar“

Es funktioniert. Und zählt trotzdem falsch. Jede Zahl in einer Datentabelle hatte einmal ein Messgerät, eine Genauigkeit und ein Ausfallrisiko.

Zu Beginn lernt der Roboter, wie weit die Wand gegenüber entfernt ist. Im Probelauf hat er zwei Durchgänge nicht einordnen können und als „unklar“ notiert. In Vorlesung 08 sehen wir eine solche Messung live auf einem Mikrocontroller.

Ihre Begriffe für die Kurslandkarte

Die Kurslandkarte wächst mit dem Semester, und Sie bestimmen mit, was hineinkommt. Ihre Antworten sammeln wir im Hörsaal gemeinsam; nach der Vorlesung werden die Begriffe dieser Woche in der Karte freigeschaltet.

Welche Begriffe aus der heutigen Vorlesung gehören für Sie in die Kurslandkarte?

Texteingabe …

Wo könnten Sie das, was heute dran war, in Ihrem Fach anwenden?

Texteingabe …

Nächste Woche

Sie bekommen die Datei des Personenzählers und die Fichtelberg-Daten — und versuchen, drei Fragen mit einer Tabellenkalkulation zu beantworten. Ohne eine Zeile Code.

Bringen Sie einen Laptop mit einer Tabellenkalkulation mit — LibreOffice Calc, Excel oder Google Sheets.